



Rzeszów, 27.05.2024

Sztuczna Inteligencja

Raport z wykonania projektu:

Porównanie algorytmów wstecznej propagacji błędu oraz Learning Vector Quantization jako klasyfikatorów danych ze zbioru „Connectionist Bench (Sonar, Mines vs. Rocks)”

Autor:

Bartłomiej Mazurkiewicz

173670

2 EF-DI L05

Spis treści

1	Wstęp	3
2	Dane	3
3	Algorytmy.....	5
3.1	Model neuronu	5
3.2	Funkcja aktywacji	6
3.3	Sieć neuronowa jednowarstwowa.....	7
3.4	Sieć neuronowa wielowarstwowa	8
3.5	Wsteczna propagacja błędu	9
3.6	Learning Vector Quantization	15
4	Badania	18
4.1	Eksperyment I	18
4.1.1	Faza 1.	18
4.1.2	Faza 2.	20
4.1.3	Faza 3.	22
4.2	Eksperyment II	24
4.3	Eksperyment III	25
4.4	Eksperyment IV	27
4.5	Eksperyment V	28
4.6	Eksperyment VI	30
5	Podsumowanie i wnioski.....	31
6	Bibliografia	33

1 Wstęp

Celem projektu była implementacja i porównanie sieci neuronowych oraz ocena skuteczności różnych technik uczenia maszynowego stosowanych do klasyfikacji danych ze zbioru. Projekt obejmuje algorytmy wstecznej propagacji błędu oraz Learning Vector Quantization (LVQ).

Kluczowym aspektem badania była analiza efektywności poszczególnych algorytmów pod kątem ich zdolności do klasyfikacji danych z podanego zbioru. W ramach analizy zostały zbadane różne parametry i czynniki wpływające na jakość klasyfikacji, takie jak: liczba neuronów, współczynnik uczenia, a także liczba epok. Wykonane badania i wykresy zostały stworzone za pomocą języka Python.

2 Dane

Plik „sonar.mines” zawierał 111 wzorców, które zostały uzyskane poprzez odbicie sygnału sonaru od metalowego cylindra pod różnymi kątami. Plik „sonar.rock” zawierał 97 wzorców uzyskanych od skał w podobnych warunkach. Każdy wzorzec był zestawem 60 liczb z zakresu od 0 do 1.

Wykorzystany do testów plik „sonar.all-data” był połączeniem dwóch poprzednich plików z odpowiadającymi im etykietami. Rekord zawierający literę „R” oznacza skałę, natomiast litera „M” przy rekordzie oznacza metalowy cylinder, czyli minę. Cały zestaw był kompletny i nie zawierał żadnych pominiętych danych.

Przed badaniem algorytmów zostały zamienione etykiety danych z tekstowych na liczbowe w następujący sposób: „R” zostało oznaczone jako 1, a etykieta „M” została oznaczona jako 2.

```
import hickle as hkl
import numpy as np
import matplotlib.pyplot as plt

filename = 'sonar.all-data'
data = np.loadtxt(filename, delimiter=',', dtype=str)
x = data[:, 0:-1].astype(float).T
y_t = data[:, -1].astype(float)
y_t = y_t.reshape(1, y_t.shape[0])

print(np.transpose([np.array(range(1, x.shape[0] + 1)), x.min(axis=1), x.max(axis=1)]))

x_min = x.min(axis=1).reshape(-1, 1)
x_max = x.max(axis=1).reshape(-1, 1)
```

```

x_norm_max = 1
x_norm_min = -1
x_norm = (x_norm_max - x_norm_min) * (x - x_min) / (x_max - x_min) + x_norm_min
print(np.transpose([np.array(range(1, x.shape[0] + 1)), x_norm.min(axis=1),
x_norm.max(axis=1)]))
for i in range(x.shape[0]):
    x_norm[i, :] = (x_norm_max - x_norm_min) / (x_max[i] - x_min[i]) * \
        (x[i, :] - x_min[i]) + x_norm_min
print(np.transpose([np.array(range(1, x.shape[0] + 1)), x_norm.min(axis=1),
x_norm.max(axis=1)]))
plt.plot(y_t[0])
hkl.dump([x, y_t, x_norm], 'sonar-zadanie.hkl')

```

Listing 2.1 Kod wczytujący i normalizujący dane

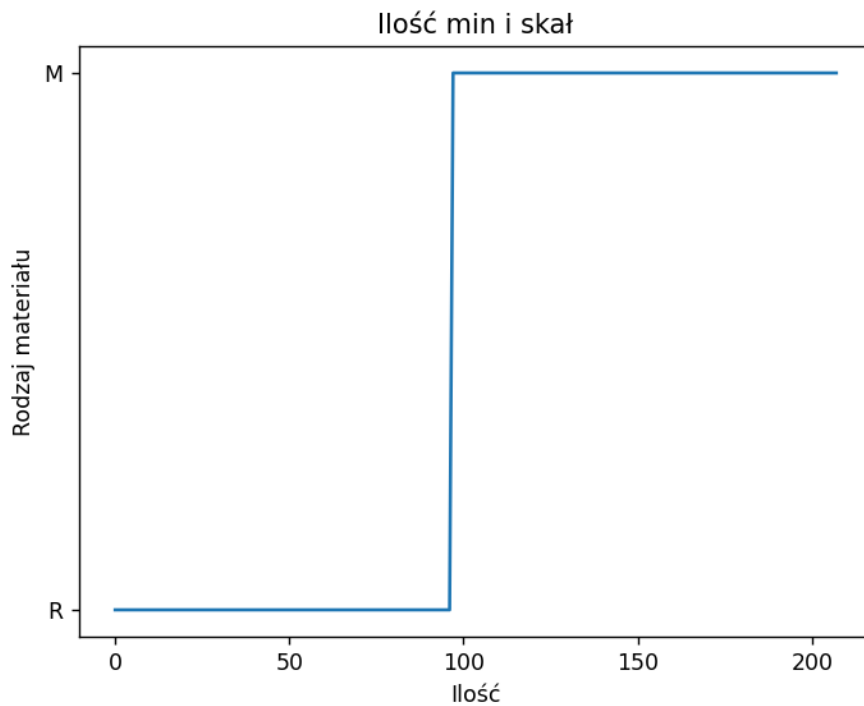
Algorytm podany w listingu 2.1 wczytywał dane z pliku „sonar.all-data” przy użyciu funkcji „np.loadtxt()”, która wczytuje plik tekstowy. Dane zostały rozdzielone przecinkami i wczytywane były jako typ „str”. Wybrane zostały kolumny od pierwszej do przedostatniej jako macierz „x”, następnie konwertowane na typ „float” i końcowo zostały transponowane. W kolejnym kroku wybrana została kolumna ostatnia z podanego zbioru i zapisywana jako macierz „y_t”, która oznaczała etykiety danych w postaci liczbowej. W następnym kroku macierz ta została przekonwertowana na typ „float”. Obliczone zostały minimalne i maksymalne wartości dla każdej cechy w macierzy „x”. Określone zostały także docelowe minimalne i maksymalne wartości dla normalizowanych danych. W kolejnym kroku dane wejściowe „x” zostały normalizowane do zakresu od -1 do 1 za pomocą wzoru:

$$x_{\text{norm}} = \frac{(x_{\text{norm}_{\text{max}}} - x_{\text{norm}_{\text{min}}}) * (x - x_{\text{min}})}{x_{\text{max}} - x_{\text{min}}} + x_{\text{norm}_{\text{min}}} \quad (1)$$

Gdzie:

- x_{norm} – dane po normalizacji
- $x_{\text{norm}_{\text{max}}}$ – maksymalny zakres normalizacji: 1
- $x_{\text{norm}_{\text{min}}}$ – minimalny zakres normalizacji: -1
- x – wartość przed normalizacją
- x_{min} – minimalna wartość w zbiorze
- x_{max} – maksymalna wartość w zbiorze

Macierze „x”, „y_t”, „x_norm” zostały zapisane do pliku „sonar-zadanie.hkl” za pomocą modułu hickle. Został również stworzony wykres na podstawie pierwszego wiersza „y_t”, który przedstawia dane w podanym zbiorze.

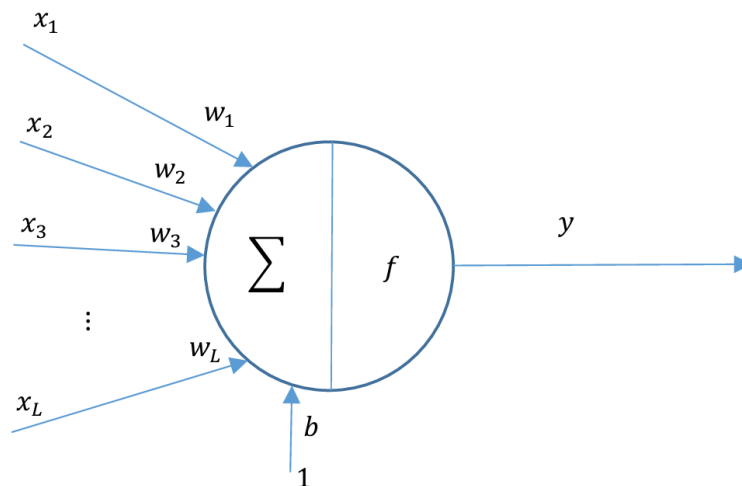


Rysunek 2.1 Przedstawienie ilości i rodzaju danych

Na rysunku 2.1 można zobaczyć, że dane w zbiorze przedstawione zostały jako miny i skały wraz z przybliżoną ich wartością. Etykieta „R” oznacza skałę, których ilość była mniejsza niż 100, natomiast etykieta „M” oznacza minę, których ilość przekracza 100 wzorców.

3 Algorytmy

3.1 Model neuronu



Rysunek 3.1 Model neuronu

Neuron jest podstawową jednostką przetwarzającą informacje w sieciach neuronowych.

Skład się on z:

- wyjścia y
- funkcji aktywacji f
- bloku sumującego Σ
- wartości przesunięcia b (bias)
- zbioru wag w_L , które powiązane są z wejściami
- zbioru wejść x_L

Sygnał wyjściowy neuronu jest określony jako zależność:

$$y = f\left(\sum_{j=1}^L w_j x_j + b\right) \quad (2)$$

Oznaczenie:

- W_j oraz x_j oznaczają odpowiednio wagi wejściowe neuronu oraz wejścia, których ilość jest równa od 1 do L.
- B oznacza bias, czyli przesunięcie.
- F z kolei oznacza funkcję aktywacji.
- Y oznacza sygnał wyjściowy

Neuron przyjmuje wartości wejściowe, mnożone przez wagi, które są później sumowane, a następnie wykorzystywane w funkcji aktywacji. Funkcja ta umożliwia sieci neuronowej modelowanie złożonych relacji między danymi wejściowymi a wyjściowymi.

3.2 Funkcja aktywacji

Funkcja aktywacji w sieci neuronowej to funkcja, która wykorzystywana jest na wyjściu każdego neuronu w sieci, oprócz neuronów warstwy wyjściowej. Funkcja aktywacji przyjmuje jako argument sumę ważonych wejść neuronu i zwraca wynik, który przekazywany jest do kolejnych neuronów lub jako wynik końcowy sieci.

Funkcja aktywacji może być między innymi:

- sigmoidalna

$$f(x) = \frac{1}{1+e^{-x}} \quad (3)$$

- tangens

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (4)$$

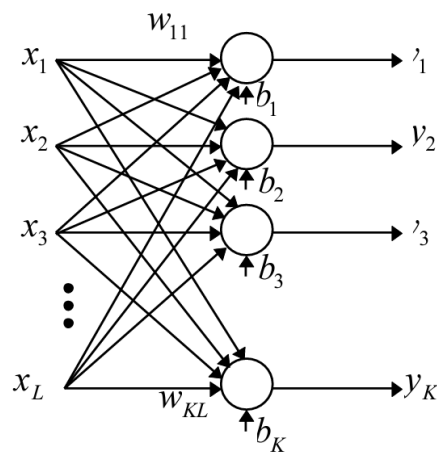
- liniowa

$$f(x) = x \quad (5)$$

Wartość „x” w podanych zależnościach (3), (4) oraz (5) oznacza wartości wejściowe.

3.3 Sieć neuronowa jednowarstwowa

Sieć neuronowa jednowarstwowa to rodzaj prostego modelu sieci neuronowej składający się z jednej warstwy neuronów. Warstwa ta jest nazywana warstwą ukrytą, ponieważ neurony te wykonują przetwarzanie bezpośrednie (nie są ani wejściami, ani wyjściami). Neurony otrzymują dane na wejściu i bezpośrednio podają wynik na wyjście.



Rysunek 3.2 Sieć neuronowa jednowarstwowa

Wejściem każdego neuronu jest wektor sygnałów $x = (x_1, x_2, \dots, x_L)^T$. Jest to połączenie każdego wejścia z każdym neuronem. Każdy neuron posiada wektor wag oznaczony w . W tym przypadku wagi te mają dodatkowy indeks, od w_{11} do w_{KL} . L oznacza liczbę wejść sieci, natomiast K oznacza ilość neuronów.

Stosując zapis macierzowy możemy opisać funkcjonowanie warstwy neuronów za pomocą:

$$y = f(wx + b) \quad (6)$$

Gdzie:

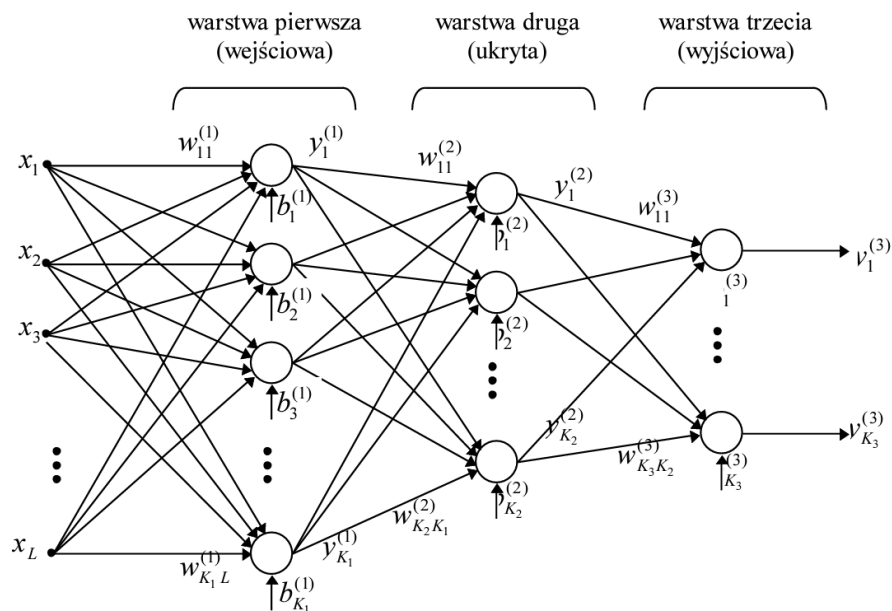
- b - oznacza bias, wektor przesunięć,
- y - to sygnały wyjściowe,
- w - jest prostokątną macierzą wag o wymiarach $K \times L$, gdzie
 - K – liczba neuronów,
 - L – liczba sygnałów wejściowych.

$$w = \begin{bmatrix} w_{11} & \cdots & w_{1L} \\ \vdots & \ddots & \vdots \\ w_{K1} & \cdots & w_{KL} \end{bmatrix} \quad (7)$$

$$b = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_K \end{bmatrix}, \quad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_K \end{bmatrix} \quad (8)$$

3.4 Sieć neuronowa wielowarstwowa

Sieć neuronowa wielowarstwowa to rodzaj sieci neuronowej składający się z co najmniej trzech warstw neuronów: warstwy wejściowej, jednej lub więcej warstw ukrytych oraz warstwy wyjściowej.



Rysunek 3.3 Sieć neuronowa wielowarstwowa

Występują tu połączenia między warstwami neuronów typu każdy z każdym. Każda warstwa neuronów posiada macierz w , czyli macierz wag, wektor przesunięć b , funkcję aktywacji f oraz wektor sygnałów wyjściowych y .

Aby obliczyć sygnał wyjściowy wykorzystujemy wzór dla wyjścia pojedynczej warstwy:

$$y^{(n)} = f^{(n)}(z^{(n)}) = f^{(n)}(w^{(n)}y^{(n-1)} + b^{(n)}) \quad (9)$$

Możemy zatem rozpatrywać każdą warstwę oddzielnie, wykorzystując następujące równania:

$$y^{(1)} = f^{(1)}(z^{(1)}) = f^{(1)}(w^{(1)}x + b^{(1)}) \quad (10)$$

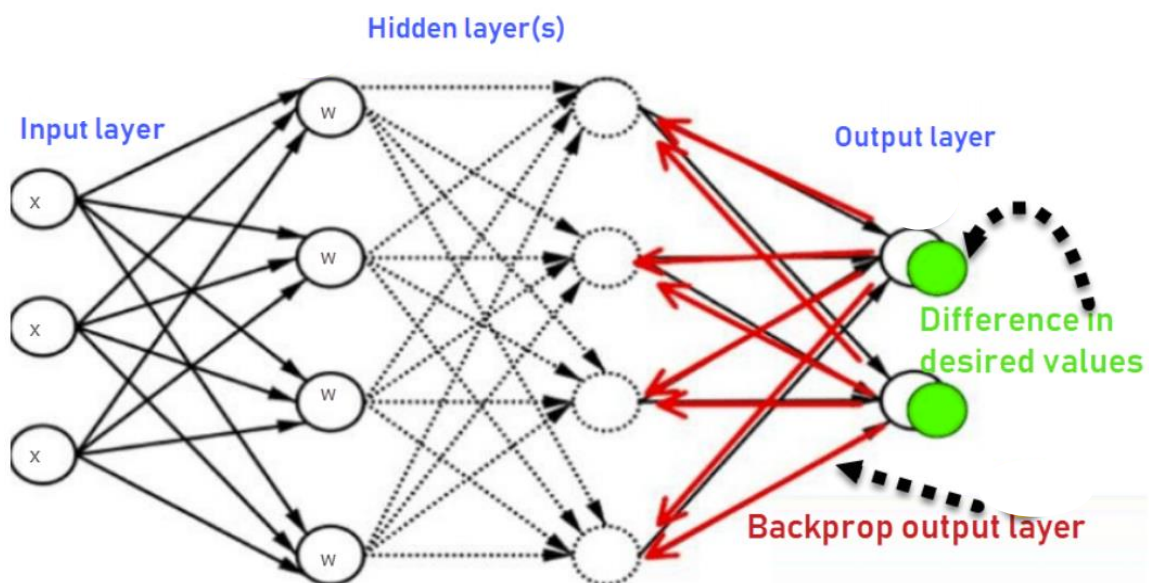
$$y^{(2)} = f^{(2)}(z^{(2)}) = f^{(2)}(w^{(2)}y^{(1)} + b^{(2)}) \quad (11)$$

$$y^{(3)} = f^{(3)}(z^{(3)}) = f^{(3)}(w^{(3)}y^{(2)} + b^{(3)}) \quad (12)$$

Zapis całej sieci trójwarstwowej, łącząc powyższe wzory, możemy zapisać równaniem:

$$y^{(3)} = f^{(3)}(w^{(3)}f^{(2)}(w^{(2)}f^{(1)}(w^{(1)}x + b^{(1)}) + b^{(2)}) + b^{(3)}) \quad (13)$$

3.5 Wsteczna propagacja błędów



Rysunek 3.4 Przedstawienie propagacji wstecznej

Algorytm wstecznej propagacji błędu to proces, który umożliwia skuteczną optymalizację wag sieci poprzez minimalizację funkcji kosztu za pomocą obliczania gradientów.

Proces rozpoczyna się od przekazania danych wejściowych przez sieć neuronową. W każdej warstwie sieci obliczane są wartości aktywacji na podstawie danych wejściowych, wag, połączeń między neuronami i biasów. Wyjście sieci porównywane jest z oczekiwanymi wartościami, a następnie rozpoczyna się przejście wsteczne. Błąd, który jest obliczany na wyjściu sieci, jest propagowany wstecz, zaczynając od ostatniej warstwy. Dla każdej warstwy obliczane są gradienty funkcji kosztu względem wag i biasów przy użyciu różniczkowania funkcji złożonej. Ostatecznie stosowany jest algorytm optymalizacji do aktualizacji wag sieci w kierunku minimalizacji błędu.

W przypadku sieci trójwarstwowej, po uwzględnieniu wszystkich zależności, błąd przyjmie postać:

$$\begin{aligned}
 y &= \frac{1}{2} \sum_{i_3=1}^{K_3} e_{i_3}^2 = \frac{1}{2} \sum_{i_3=1}^{K_3} (y_{i_3}^{(3)} - \widehat{y}_{i_3})^2 \\
 &= \frac{1}{2} \sum_{i_3=1}^{K_3} \left(f^{(3)} \left(\sum_{i_2=1}^{K_2} w_{i_3 i_2}^{(3)} y_{i_2}^{(2)} + b_{i_3}^{(3)} \right) - \widehat{y}_{i_3} \right)^2 \\
 &= \frac{1}{2} \sum_{i_3=1}^{K_3} \left(f^{(3)} \left(\sum_{i_2=1}^{K_2} w_{i_3 i_2}^{(3)} f^{(2)} \left(\sum_{i_1=1}^{K_1} w_{i_2 i_1}^{(2)} y_{i_1}^{(1)} + b_{i_2}^{(2)} \right) + b_{i_3}^{(3)} \right) - \widehat{y}_{i_3} \right)^2 \\
 &= \frac{1}{2} \sum_{i_3=1}^{K_3} \left(f^{(3)} \left(\sum_{i_2=1}^{K_2} w_{i_3 i_2}^{(3)} f^{(2)} \left(\sum_{i_1=1}^{K_1} w_{i_2 i_1}^{(2)} f^{(1)} \left(\sum_{j=1}^L w_{i_1 j}^{(1)} x_j + b_{i_1}^{(1)} \right) + b_{i_2}^{(2)} \right) + b_{i_3}^{(3)} \right) - \widehat{y}_{i_3} \right)^2
 \end{aligned}
 \tag{14}$$

Gdzie:

- $j=1, \dots, L$ – numer wejścia warstwy pierwszej
- $j_1 = 1, \dots, K_1; j_2 = 1, \dots, K_2; j_3 = 1, \dots, K_3$ – numery wyjść warstw pierwszej, drugiej i trzeciej

Wagi neuronów w poszczególnych warstwach określa wzór:

$$\Delta w_{ij} = \eta \frac{\partial E}{\partial w_{ij}} \quad (15)$$

Gdzie:

- Δw_{ij} – zmiana wagi między neuronem i w jednej warstwie, a neuronem j w kolejnej warstwie
- η – współczynnik uczenia
- $\frac{\partial E}{\partial w_{ij}}$ – pochodna cząstkowa funkcji błędu E względem wag w_{ij}

Obliczanie wag neuronów rozpoczyna się od warstwy wyjściowej, dla której mamy:

$$\frac{\partial E}{\partial w_{i_3 i_2}^{(3)}} = \frac{\partial E}{\partial y_{i_3}^{(3)}} \frac{\partial y_{i_3}^{(3)}(z_{i_3}^{(3)})}{\partial z_{i_3}^{(3)}} \frac{\partial z_{i_3}^{(3)}}{\partial w_{i_3 i_2}^{(3)}} = (y_{i_3}^{(3)} - \widehat{y}_{i_3}) \frac{\partial y_{i_3}^{(3)}(z_{i_3}^{(3)})}{\partial z_{i_3}^{(3)}} y_{i_2}^{(2)} \quad (16)$$

$$z_{i_3}^{(3)} = \sum_{i_2=1}^{K_2} w_{i_3 i_2}^{(3)} y_{i_2}^{(2)} + b_{i_3}^{(3)} \quad (17)$$

Zależność (17) to pobudzenie i_3 – tego neuronu warstwy wyjściowej.

W analogiczny sposób wyznaczamy elementy gradientu względem wag warstwy ukrytej i wejściowej.

$$\begin{aligned} \frac{\partial E}{\partial w_{i_2 i_1}^{(2)}} &= \frac{\partial E}{\partial y_{i_3}^{(3)}} \frac{\partial y_{i_3}^{(3)}(z_{i_3}^{(3)})}{\partial z_{i_3}^{(3)}} \frac{\partial z_{i_3}^{(3)}}{\partial y_{i_2}^{(2)}} \frac{\partial y_{i_2}^{(2)}(z_{i_2}^{(2)})}{\partial z_{i_2}^{(2)}} \frac{\partial z_{i_2}^{(2)}}{\partial w_{i_2 i_1}^{(2)}} \\ &= \sum_{i_3=1}^{K_3} (y_{i_3}^{(3)} - \widehat{y}_{i_3}) \frac{\partial y_{i_3}^{(3)}(z_{i_3}^{(3)})}{\partial z_{i_3}^{(3)}} w_{i_3 i_2}^{(3)} \frac{\partial y_{i_2}^{(2)}(z_{i_2}^{(2)})}{\partial z_{i_2}^{(2)}} y_{i_1}^{(1)} \end{aligned} \quad (18)$$

$$\begin{aligned}
\frac{\partial E}{\partial w_{i_1 j}^{(1)}} &= \frac{\partial E}{\partial y_{i_3}^{(3)}} \frac{\partial y_{i_3}^{(3)}(z_{i_3}^{(3)})}{\partial z_{i_3}^{(3)}} \frac{\partial z_{i_3}^{(3)}}{\partial y_{i_2}^{(2)}} \frac{\partial y_{i_2}^{(2)}(z_{i_2}^{(2)})}{\partial z_{i_2}^{(2)}} \frac{\partial z_{i_2}^{(2)}}{\partial y_{i_1}^{(1)}} \frac{\partial y_{i_1}^{(1)}(z_{i_1}^{(1)})}{\partial z_{i_1}^{(1)}} \frac{\partial z_{i_1}^{(1)}}{\partial w_{i_1 j}^{(1)}} \\
&= \sum_{i_3=1}^{K_3} (y_{i_3}^{(3)} - \hat{y}_{i_3}) \frac{\partial y_{i_3}^{(3)}(z_{i_3}^{(3)})}{\partial z_{i_3}^{(3)}} \sum_{i_2=1}^{K_2} \frac{\partial y_{i_2}^{(2)}(z_{i_2}^{(2)})}{\partial z_{i_2}^{(2)}} w_{i_2 i_1}^{(2)} \frac{\partial y_{i_1}^{(1)}(z_{i_1}^{(1)})}{\partial z_{i_1}^{(1)}} x_j
\end{aligned}$$

(19)

```

import hickle as hkl
import numpy as np
import nnet as net
import matplotlib.pyplot as plt
from sklearn.model_selection import StratifiedKFold
from timeit import default_timer as timer
class mlp__3w:
    def __init__(self, x, y_t, K1, K2, lr, err_goal, \
        disp_freq, max_epoch):
        self.x = x
        self.L = self.x.shape[0]
        self.y_t = y_t
        self.K1 = K1
        self.K2 = K2
        self.lr = lr
        self.err_goal = err_goal
        self.disp_freq = disp_freq
        self.max_epoch = max_epoch
        self.K3 = y_t.shape[0] if len(y_t.shape)>1 else 1
        self.SSE_vec = []
        self.PK_vec = []
        self.w1, self.b1 = net.nwtan(self.K1, self.L)
        self.w2, self.b2 = net.nwtan(self.K2, self.K1)
        self.w3, self.b3 = net.rands(self.K3, self.K2)
        self.SSE = 0
        self.lr_vec = list()
    def predict(self,x):
        n = np.dot(self.w1, x)
        self.y1 = net.tansig( n, self.b1*np.ones(n.shape))
        n = np.dot(self.w2, self.y1)
        self.y2 = net.tansig( n, self.b2*np.ones(n.shape))
        n = np.dot(self.w3, self.y2)
        self.y3 = net.purelin(n, self.b3*np.ones(n.shape))
        return self.y3

```

```

def train(self, x_train, y_train):
    for epoch in range(1, self.max_epoch+1):
        self.y3 = self.predict(x_train)
        self.e = y_train - self.y3
        self.SSE_t_1 = self.SSE
        self.SSE = net.sumsqr(self.e)
        self.PK = (1 - sum((abs(self.e)>=0.5).astype(int)[0])/self.e.shape[1]) * 100
        self.PK_vec.append(self.PK)
        if self.SSE < self.err_goal or self.PK == 100:
            break
        if np.isnan(self.SSE):
            break

        self.d3 = net.deltalin(self.y3, self.e)
        self.d2 = net.deltatan(self.y2, self.d3, self.w3)
        self.d1 = net.deltatan(self.y1, self.d2, self.w2)
        self.dw1, self.db1 = net.learnbp(self.x, self.d1, self.lr)
        self.dw2, self.db2 = net.learnbp(self.y1, self.d2, self.lr)
        self.dw3, self.db3 = net.learnbp(self.y2, self.d3, self.lr)
        self.w1 += self.dw1
        self.b1 += self.db1
        self.w2 += self.dw2
        self.b2 += self.db2
        self.w3 += self.dw3
        self.b3 += self.db3
        self.SSE_vec.append(self.SSE)
x_nie_norm, y_t, x = hkl.load('sonar-zadanie.hkl')
max_epoch = 1000
err_goal = 0.25
disp_freq = 10
lr = 0.0001

K1 = 75
K2 = 5
data = x.T
target = y_t
CVN = 20
skfold = StratifiedKFold(n_splits=CVN)
PK_vec = np.zeros(CVN)
start = timer()
for i, (train, test) in enumerate(skfold.split(data, np.squeeze(target)), start=0):
    x_train, x_test = data[train], data[test]
    y_train, y_test = np.squeeze(target)[train], np.squeeze(target)[test]
    mlpnet = mlp__3w(x_train.T, y_train, K1, K2, lr, err_goal, \
                    disp_freq, max_epoch)
    mlpnet.train(x_train.T, y_train.T)
    result = mlpnet.predict(x_test.T)
    n_test_samples = test.size
    PK_vec[i] = (1 - sum((abs(result - y_test)>=0.5).astype(int)[0])/n_test_samples) * 100
    print("Test #{:<2}: PK_vec {} test_size {}".format(i, PK_vec[i], n_test_samples))
print("Execution time:", timer()-start)
PK = np.mean(PK_vec)
print("PK {}".format(PK))

```

```

K1 = 75
K2 = 5
data = x.T
target = y_t
CVN = 20

skfold = StratifiedKFold(n_splits=CVN)
PK_vec = np.zeros(CVN)
start = timer()
for i, (train, test) in enumerate(skfold.split(data, np.squeeze(target)), start=0):
    x_train, x_test = data[train], data[test]
    y_train, y_test = np.squeeze(target)[train], np.squeeze(target)[test]
    mlpnet = mlp__3w(x_train.T, y_train, K1, K2, lr, err_goal, \
                     disp_freq, max_epoch)
    mlpnet.train(x_train.T, y_train.T)
    result = mlpnet.predict(x_test.T)
    n_test_samples = test.size
    PK_vec[i] = (1 - sum((abs(result - y_test)>=0.5).astype(int)[0])/n_test_samples) * 100
    print("Test #{<2}: PK_vec {} test_size {}".format(i, PK_vec[i], n_test_samples))
print("Execution time:", timer()-start)
PK = np.mean(PK_vec)
print("PK {}".format(PK))

```

Listing 3.1 Wsteczna propagacja błędu

Na początku, w klasie „mlp__3w” inicjalizowane są różne parametry i zmienne, takie jak: dane wejściowe, oczekiwane dane wyjścia, liczba neuronów w warstwach ukrytych, współczynnik uczenia, współczynnik błędu, częstotliwość wyświetlania, maksymalna liczba epok. Następnie w funkcji „predict” obliczane jest wyjście sieci dla danego wejścia. Funkcja ta wykorzystuje wagi i biasy, aby obliczyć sumę ważoną wejść dla każdego neuronu w warstwie ukrytej. Dla każdego z tych neuronów stosowana jest funkcja aktywacji, w tym przypadku „tansig”, która przekształca sumy ważone na wartości wyjściowe neuronów. Wynik z warstw ukrytych są przekazywane do kolejnej warstwy sieci. Funkcja „predict” oblicza także sumę ważoną dla neuronów w warstwie wyjściowej, a następnie stosuje funkcję aktywacji, gdzie w tym przypadku jest to „pureline”. Ostatecznie wartości obliczone dla warstwy wyjściowej są zwracane jako wynik funkcji „predict”, który reprezentuje wyjście sieci dla danego zestawu danych wejściowych.

Funkcja „train” służy do uczenia sieci neuronowej. Jest odpowiedzialna za iteracyjne dostosowywanie wag i biasów w celu minimalizacji błędu predykcji. Rozpoczyna się pętlą treningową, która iteruje przez określoną liczbę epok lub do momentu osiągnięcia założonego błędu. Funkcja wywołuje funkcję „predict”, aby obliczyć wyjście sieci dla aktualnego zestawu danych treningowych. Następnie oblicza błąd predykcji, który jest propagowany wstecz przez sieć, aby obliczyć wpływ błędu na każdą warstwę i każdy neuron. Po obliczeniu gradientów funkcji błędu, funkcja „train” wykonuje aktualizację, by dostosować parametry wag i biasów. Po każdej aktualizacji sprawdza, czy warunek stopu został spełniony.

W kolejnej części kodu wczytywane są dane wejściowe do programu, definiowane są parametry treningowe, a następnie kod przeprowadza walidację krzyżową na ustawionej ilości podziałach danych, gdzie dla każdego podziału sieć jest trenowana na jednej części danych, a na pozostałej części jest testowana. Dla każdego podziału obliczana jest trafność klasyfikacji

sieci nazwano jako „PK”. Końcowo jest obliczana średnia „PK” dla wszystkich podziałów, co daje ogólną skuteczność sieci.

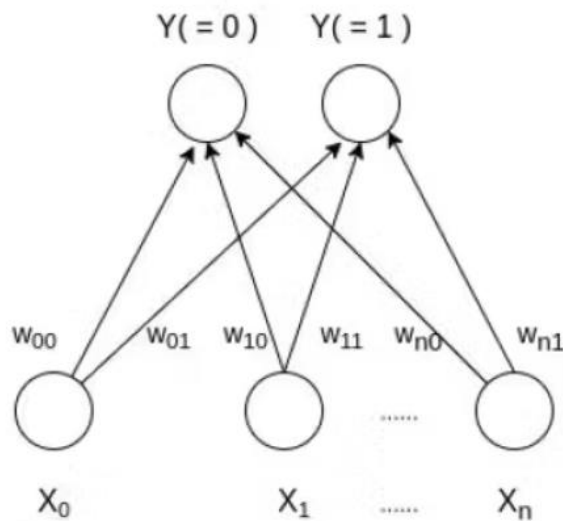
3.6 Learning Vector Quantization

Algorytm LVQ (Learning Vector Quantization) jest typem sieci neuronowej, w której neurony nazwane „wektorami kodującymi” są wektorami zdefiniowanymi na wejściu z przypisanymi etykietami klas. Liczba tych neuronów jest mniejsza, niż liczba wektorów treningowych:

$$l_w < l_x \quad (20)$$

Gdzie:

- l_w - liczba neuronów (wektorów kodujących)
- l_x - liczba wektorów treningowych



Rysunek 3.5 Sieć LVQ

Algorytm ten polega na przyciąganiu do siebie wektorów kodujących do wektorów treningowych należących do tych samych klas, a odpychania ich od wektorów o różnych klasach. W procesie uczenia, neurony (wektory kodujące) są dostosowywane iteracyjnie, aby lepiej odwzorowywać charakterystyki klas danych.

Proces uczenia algorytmu LVQ składa się z głównej pętli, która wykonuje się określoną liczbę razy. Wewnątrz niej, dla każdego elementu w zbiorze treningowym, szukamy najbliższego neuronu kodującego w przestrzeni wejściowej. Jeśli wektor treningowy należy do tej samej klasy co neuron kodujący, to aktualizowana jest waga tego neuronu zgodnie ze wzorem:

$$p_i(k+1) = p_i(k) + \alpha (x_j - p_i(k)) \quad (21)$$

Jeżeli etykiety wektora kodującego i treningowego są różne, wówczas aktualizacja wag odbywa się za pomocą następującej zależności:

$$p_i(k+1) = p_i(k) - \alpha(x_j - p_i(k)) \quad (22)$$

W trakcie iteracji współczynnik uczenia α powinien być aktualizowany według następującej zależności:

$$\alpha(k+1) = \frac{\alpha(k)}{1 + \alpha(k)} \quad (23)$$

Gdzie dla zależności (21), (22) oraz (23):

- p_i - i-ty wektor kodujący
- x_j - j-ty wektor treningowy
- k - numer iteracji
- α - współczynnik uczenia

```
import numpy as np
import hickle as hkl
import math
def lvqTraining(xt, y, lr, max_epoch, err_goal):
    SSE_vec = []
    label, train_idx = np.unique(y, return_index=True)
    W = xt[train_idx].astype(np.float64)
    filtrX = np.array([e for i, e in enumerate(xt) if i not in train_idx])
    filtrY = np.array([e for i, e in enumerate(y) if i not in train_idx])
    for epoch in range(1, max_epoch + 1):
        SSE = 0.0
        predicted_labels = []
        for i, x in enumerate(filtrX):
            d = [math.sqrt(sum((w - x) ** 2)) for w in W]
            Dmin = np.argmin(d)
            predicted_labels.append(label[Dmin])
            if filtrY[i] == label[Dmin]:
                W[Dmin] = W[Dmin] + lr * (x - W[Dmin])
            else:
                W[Dmin] = W[Dmin] - lr * (x - W[Dmin])
        SSE = net.sumsqr((W[Dmin] - x).reshape(1, -1)) / len(x)
```



```

if epoch % 10:
    SSE_vec.append(SSE)
    print("Epoch: %5d | SSE: %5.5e" % (epoch, SSE))
if SSE < err_goal or np.isnan(SSE) or np.isinf(SSE):
    print("Epoch: %5d | SSE: %5.5e" % (epoch, SSE))
    break
return W, label

def lvqTest(x, W):
    weights, label = W
    d = [math.sqrt(sum((w - x) ** 2)) for w in weights]
    return label[np.argmin(d)]

x_nie_norm, y_t, x = hkl.load('sonar-zadanie.hkl')
data = x.T
target = np.squeeze(y_t)
W = lvqTraining(data, target, lr=0.005, max_epoch=1000, err_goal=1e-4)
predicted = []
for sample in data:
    predicted.append(lvqTest(sample, W))
    # print("SAMPLE:", sample)
    # print("PREDICTED:", predicted[-1])
result = np.array(predicted)
PK = np.sum(result == target) / len(target)
print("PK {}".format(PK))

```

Listing 3.2 Learning Vector Quantization

Funkcja ‘lvqTraining’ jest odpowiedzialna za trening modelu LVQ. Na wejściu otrzymuje dane treningowe ‘x’, etykiety klas ‘y’, współczynnik uczenia ‘lr’, maksymalną liczbę epok ‘max_epoch’ oraz współczynnik błędu ‘err_goal’. Wewnątrz funkcji inicjalizowane są wagi ‘W’ na podstawie unikalnych etykiet klas oraz indeksów danych treningowych. Następnie przeprowadzana jest iteracja po danych treningowych, aktualizując wagi w zależności od najbliższego wektora wag. Funkcja zatrzymuje się, gdy osiągnięty zostanie docelowy błąd lub maksymalna liczba epok.

Funkcja ‘lvqTest’ służy do testowania wytrenowanego modelu na nowych danych. Przyjmuje dane testowe oraz wagi, a następnie oblicza odległość między danymi testowymi, a każdym wektorem wag. Wynikiem jest etykieta klasy, która została przypisana do danego testu.

W dalszej części kodu wczytywane są dane, a następnie przekazywane do ‘lvqTraining’, aby wytrenować model. Po zakończeniu treningu przeprowadzane jest testowanie modelu na tych samych danych treningowych, aby obliczyć skuteczność klasyfikacji.

4 Badania

Celem badań było przeprowadzenie analizy skuteczności różnych algorytmów oraz optymalizacji parametrów sieci neuronowych. Głównym celem było upewnienie się, że wybrane algorytmy są poprawnie zaimplementowane i przetestowane, a następnie porównanie wyników użytych algorytmów przy zastosowaniu różnych parametrów sieci. Aby osiągnąć ten cel, została przeprowadzona seria eksperymentów, w których modyfikowano różne współczynniki, takie jak współczynnik uczenia, liczba neuronów w warstwach, a także liczba epok. Dzięki tym modyfikacjom można było zbadać, jak każdy z tych parametrów wpływa na proces uczenia się sieci neuronowej, by móc je następnie porównać.

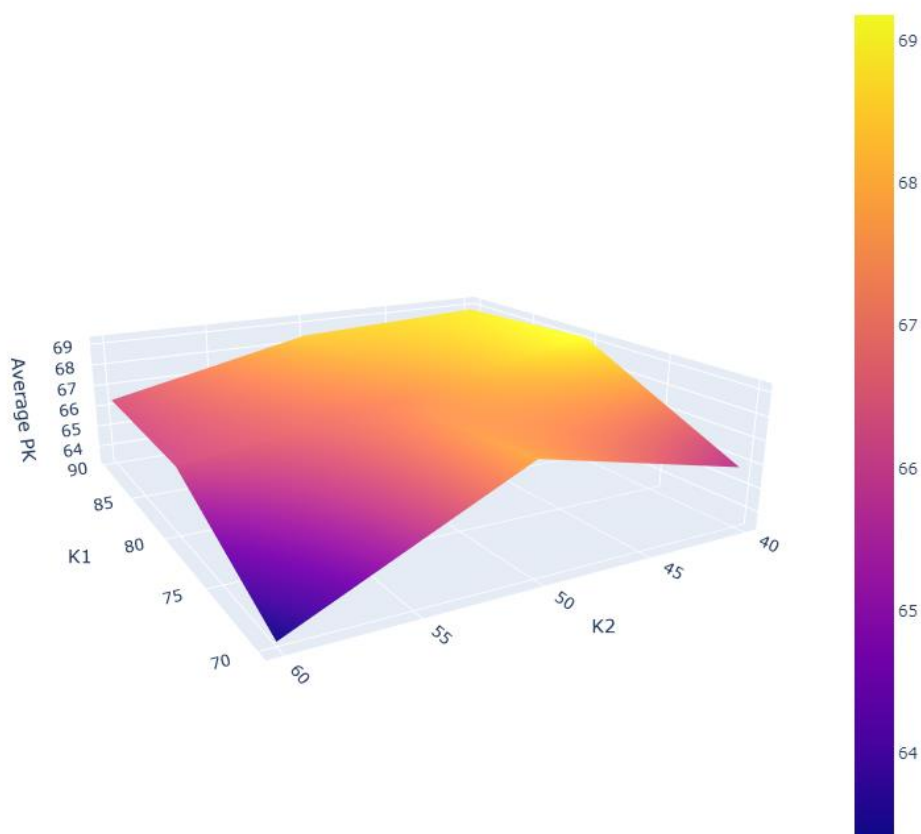
Liczba epok odnosi się do ilości pełnych przejść algorytmu przez cały zbiór danych treningowych, współczynnik błędu określa natomiast, kiedy trening modelu powinien zostać zakończony. Współczynnik uczenia to parametr, który określa wielkość kroku, jaki jest wykonywany podczas aktualizacji wag w sieci neuronowej.

4.1 Eksperyment I

W eksperymencie pierwszym wykorzystany został algorytm wstecznej propagacji błędów, gdzie założone zostały następujące parametry: liczba epok równa 1000, współczynnik błędów wynoszący $1e-10$ oraz współczynnik uczenia ustalony jako wartość 0.0001. W tym badaniu została wykazana zależność poprawności uczenia sieci przy zmiennej liczbie neuronów w warstwach ukrytych. Celem eksperymentu było znalezienie optymalnej liczby neuronów, aby sieć osiągnęła jak najlepsze wyniki, co umożliwi porównanie tego algorytmu z algorytmem LVQ.

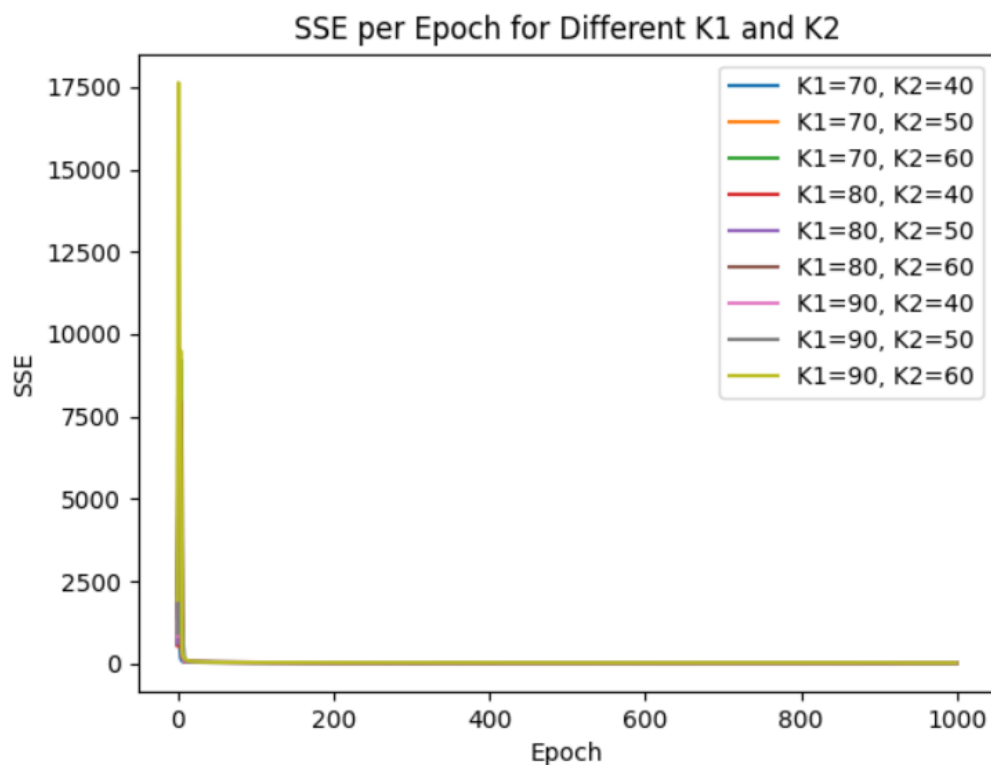
4.1.1 Faza 1.

W fazie pierwszej eksperymentu liczba neuronów została ustawiona jako $K1=[70,80,90]$ dla warstwy pierwszej oraz $K2=[40, 50, 60]$ dla warstwy drugiej. Wyniki sieci wykazały poprawność klasyfikacji w zakresie od 63% do 69%. Najlepszy rezultat uzyskano dla konfiguracji 80 neuronów w warstwie K1 oraz 40 neuronami w warstwie K2, gdzie dokładność klasyfikacji wyniosła 69%. Średnia dokładność klasyfikacji dla wszystkich testowanych konfiguracji liczby neuronów wyniosła 66%.



Rysunek 4.1 Zależność poprawności klasyfikacji od wyższej liczby neuronów

Na rysunku 4.1 widać zależność, między dokładnością klasyfikacji modelu, a liczbą neuronów w warstwach ukrytych. Można zaobserwować, że zwiększenie liczby neuronów w pierwszej warstwie ukrytej ogólnie przyczyniła się do poprawy wyników klasyfikacji, jednakże liczba neuronów w drugiej warstwie również ma istotny wpływ na końcową dokładność. Dla tego zestawu, liczba neuronów w liczbie 40 przyniosła najwyższą poprawność klasyfikacji.

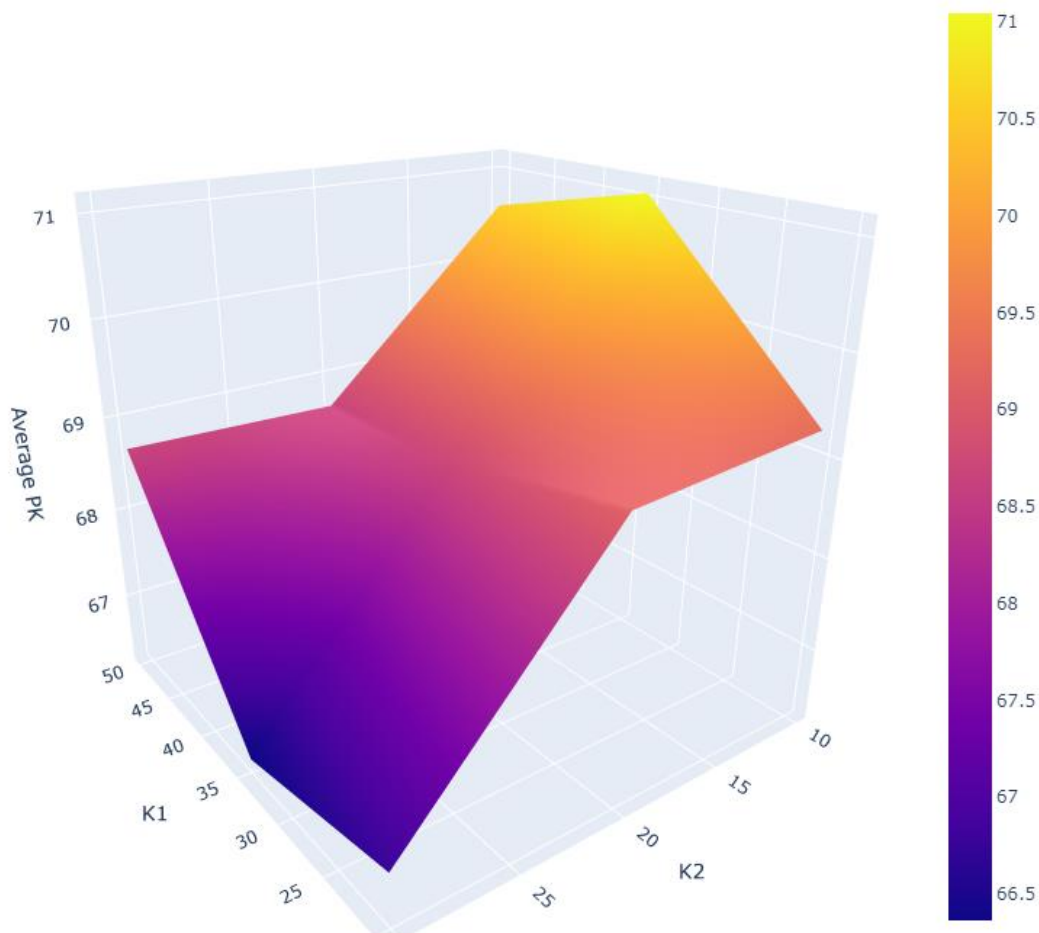


Rysunek 4.2 Wykres błędu SSE dla fazy 1 eksperymentu I

Rysunek 4.2 przedstawia wykres błędu SSE w zależności od kolejnych epok dla poszczególnej liczby neuronów w warstwach ukrytych. Można na nim zauważyć, że wartość tego błędu maleje wraz z kolejnymi epokami. Jest to zjawisko jak najbardziej oczekiwane, ponieważ świadczy o skutecznym uczeniu się sieci neuronowej. SSE, czyli suma kwadratów błędów, to różnica między wartościami rzeczywistymi, a wartościami przewidywanymi przez model.

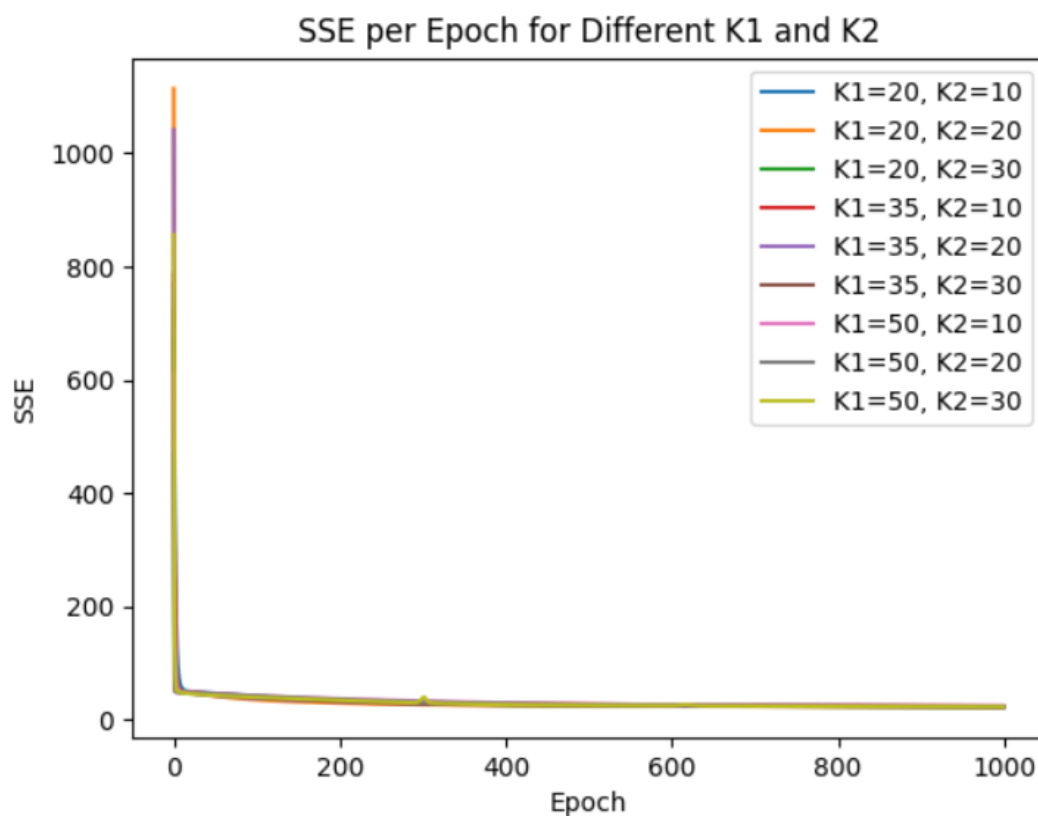
4.1.2 Faza 2.

W fazie drugiej eksperymentu przyjęto tę samą konfigurację, co w fazie pierwszej, zmieniając jedynie liczbę neuronów w warstwach ukrytych. Liczba neuronów wynosiła odpowiednio: dla warstwy $K1=[20, 35, 50]$, natomiast dla warstwy $K2=[10, 20, 30]$. Faza druga miała na celu wykazanie, czy większa liczba neuronów w warstwach ukrytych zawsze prowadzi do lepszych wyników.



Rysunek 4.3 Zależność poprawności klasyfikacji od niższej liczby neuronów

Na rysunku 4.3 widać, że średnia poprawność klasyfikacji się poprawiała i znajdowała się w przedziale od 66% do 71%. Najlepszy wynik osiągnęła dla $K1=35$ oraz $K2=10$ i wyniosła maksymalną wartość. Średnia wyników znalazła się w okolicy 68%, zatem można stwierdzić, że mniejsza liczba neuronów poprawiła wyniki modelu.

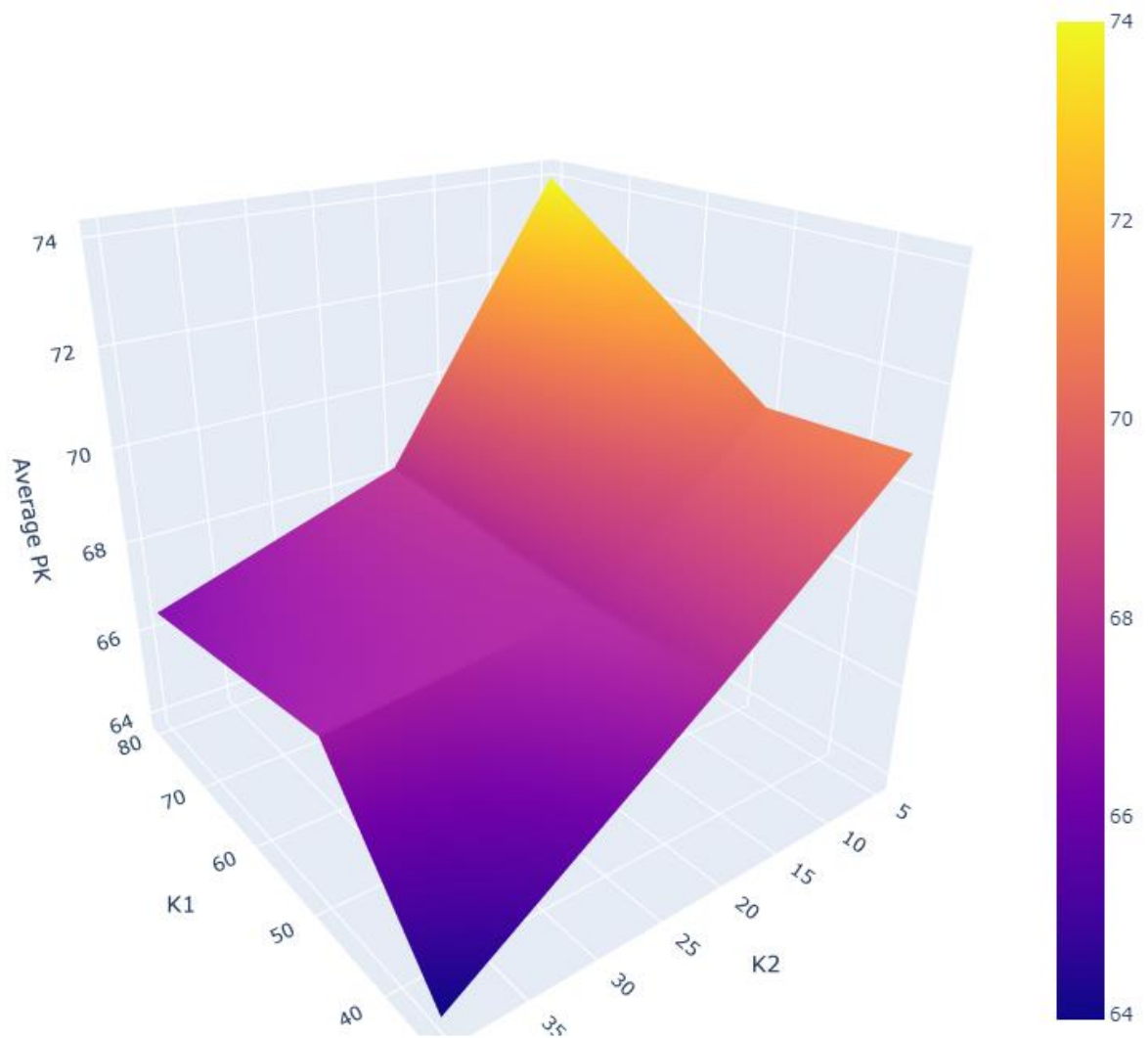


Rysunek 4.4 Wykres błędu SSE dla fazy 2 eksperymentu I

Rysunek 4.4 przedstawia wykres błędu SSE dla poszczególnej liczby neuronów w warstwach ukrytych. Podobnie jak na rysunku 4.2, można na nim zaobserwować, że wartość błędu maleje w kolejnych epokach, co jest pozytywnym zjawiskiem.

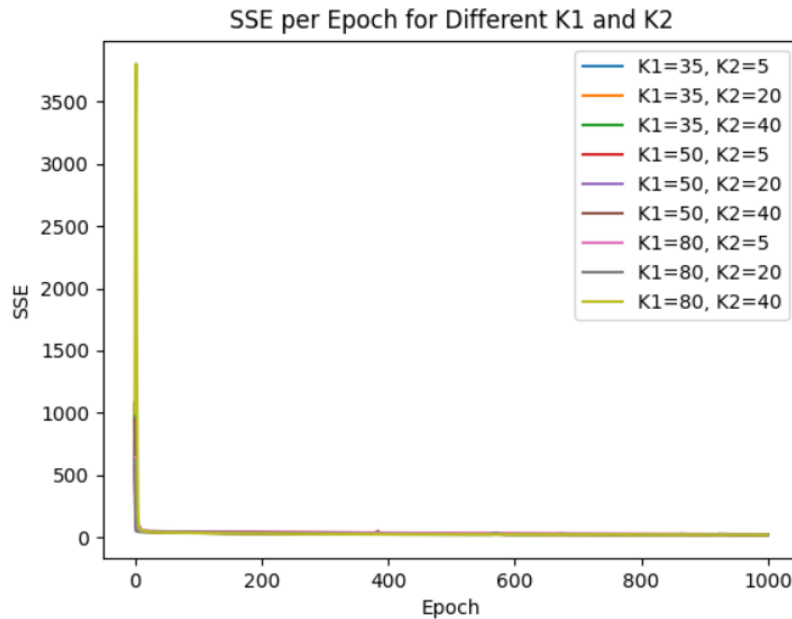
4.1.3 Faza 3.

W fazie trzeciej eksperymentu pierwszego przetestowano różne kombinacje liczb neuronów w warstwach, które wyniosły odpowiednio: dla warstwy K1=[35,50,80], a dla warstwy K2=[5,20,40]. Pozostałe współczynniki zostały utrzymane bez zmian.



Rysunek 4.5 Wpływ zróżnicowanej ilości neuronów na poprawność klasyfikacji

Na rysunku 4.5 można zauważyć, że średnia poprawność klasyfikacji się zmieniła do przedziału od 64% do 74%. Najwyższy wynik został osiągnięty dla $K1=80$ oraz $K2=5$, natomiast średnia wyników znalazła się w okolicy 69%.



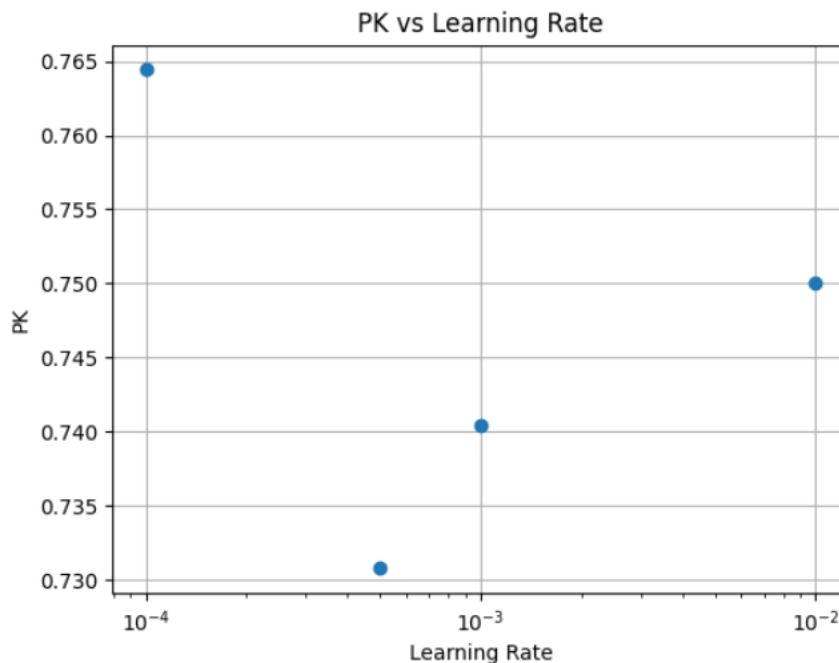
Rysunek 4.6 Wykres błędu SSE dla fazy 3 eksperymentu I

Błąd SSE w tym przypadku również zmniejsza się w oczekiwany sposób, co przedstawia rysunek 4.6. Wraz z kolejną liczbą epok, wartość błędu się zmniejsza i zbliża do zera.

Na podstawie tego eksperymentu została dobrana liczba neuronów w warstwach K1 oraz K2 i wynoszą odpowiednio: 80 i 5. Ta konfiguracja została użyta w kolejnych eksperymentach w celach porównawczych z algorytmem LVQ, gdzie badane są pozostałe współczynniki wpływające na efektywność sieci.

4.2 Eksperyment II

Eksperyment ten był przeznaczony dla oceny wpływu współczynnika uczenia na algorytm LVQ. W tym celu zostały przetestowane różne jego wartości przy ilości epok równej 500. Stworzony został wykres, na którym widać zależności między poprawnością klasyfikacji i współczynnikiem uczenia, który znajduje się na rysunku 4.7.

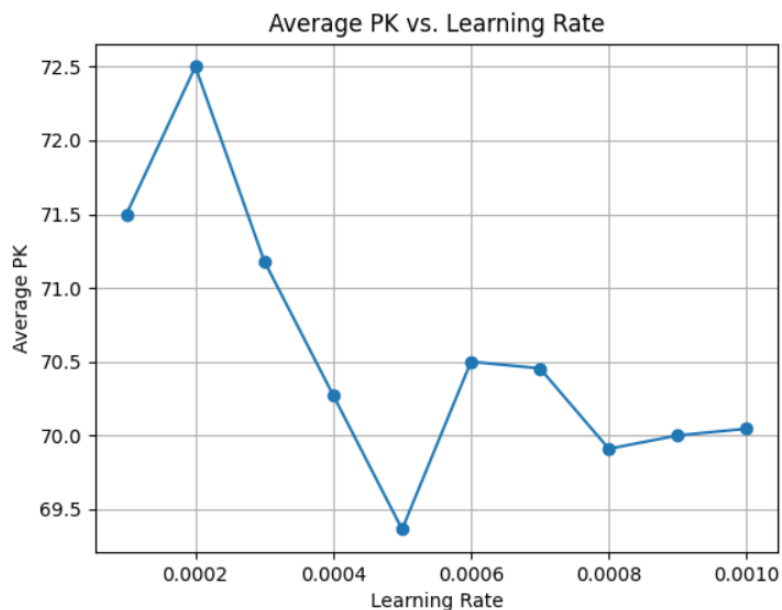


Rysunek 4.7 Wartość PK dla różnych współczynników lr w LVQ

Rysunek 4.7 wykazał, że przedział PK wynosi od 73% do 76%, gdzie minimalna wartość została osiągnięta dla współczynnika uczenia równego 0.0005, natomiast najwyższa wartość została wykazana dla lr równego 0.0001. Oznacza to, że współczynnik uczenia równy 0.0001 był najlepszą wartością dla uczenia tej sieci.

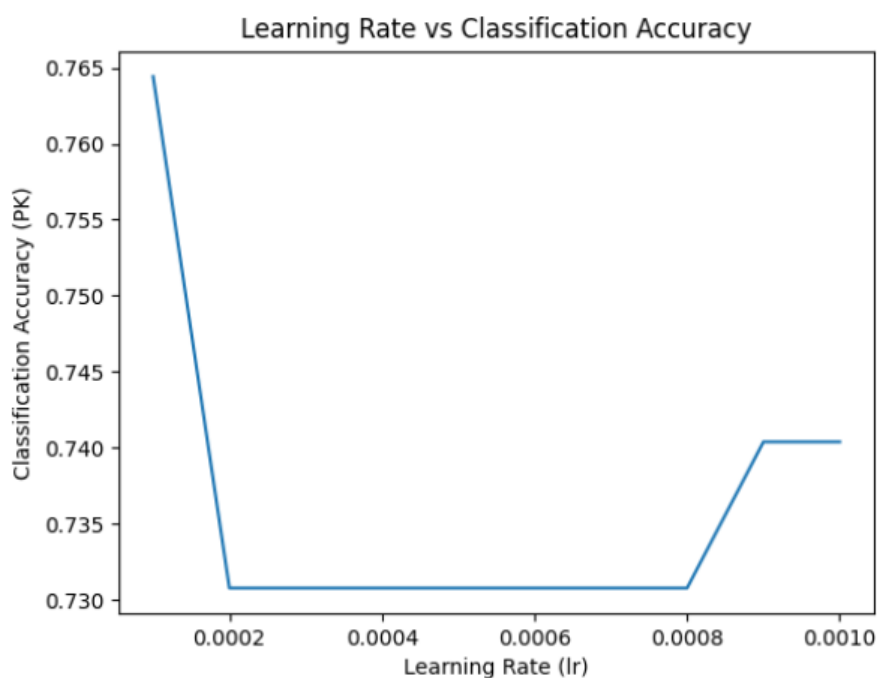
4.3 Eksperyment III

Eksperyment trzeci opierał się o porównanie algorytmu wstecznej propagacji błędu oraz Learning Vector Quantization przy różnym współczynniku uczenia. Dla pierwszego z nich przyjęto odpowiednie parametry: $K1=80$ i $K2=5$, które zostały dobrane na podstawie eksperymentu pierwszego. Dla obu algorytmów współczynnik błędu został przyjęty jako $1e-10$, natomiast liczba epok została ustawiona na 500. Współczynnik uczenia to kolejne liczby od 0.001 do 0.0001 z krokiem co 0.0001.



Rysunek 4.8 Wartość PK dla wstecznej propagacji błędów przy kolejnych lr

Rysunek 4.8 zawiera wykres średniej poprawności klasyfikacji w kolejnych wartościach współczynnika uczenia dla wstecznej propagacji błędów. Widać na nim, że najlepiej dobraną wartością została liczba 0.0002 i wartość PK osiągnęła dla niej 72%, natomiast im większy współczynnik, tym gorsze kazały się wartości klasyfikacji. Dla $lr=0.0003$ wartość osiągnęła najniższy punkt wynoszący 68% poprawności.

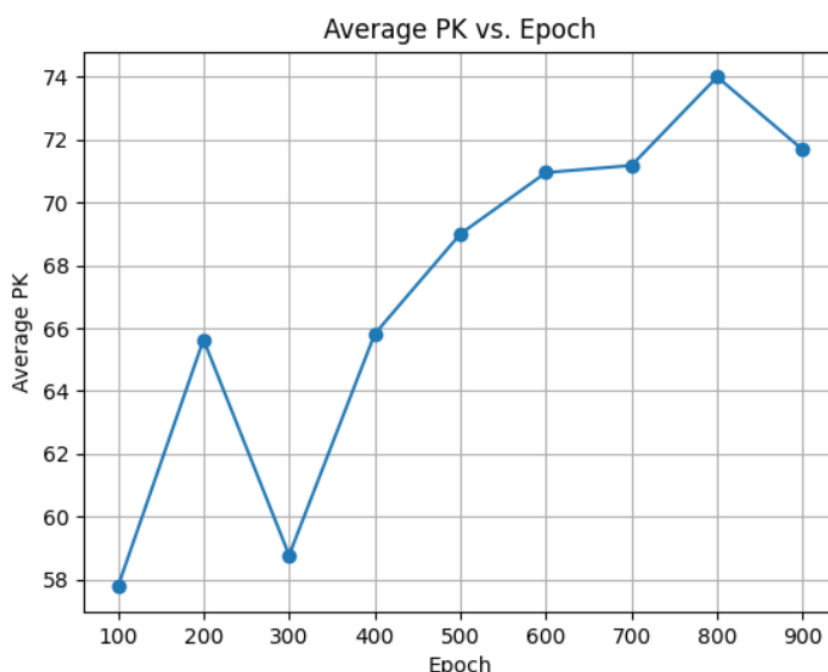


Rysunek 4.9 Wartość PK dla LVQ przy kolejnych lr

Dla algorytmu LVQ, najlepszym współczynnikiem uczenia okazała się być wartość 0.0001 i poprawności klasyfikacji dla niego wyniosła aż 76%. Dla współczynników uczenia od 0.0002 do 0.0008 PK wynosi w przybliżeniu 73%, natomiast dla $lr=0.001$ poprawność klasyfikacji wyniosła 74%. Zależności te widać na rysunku 4.9.

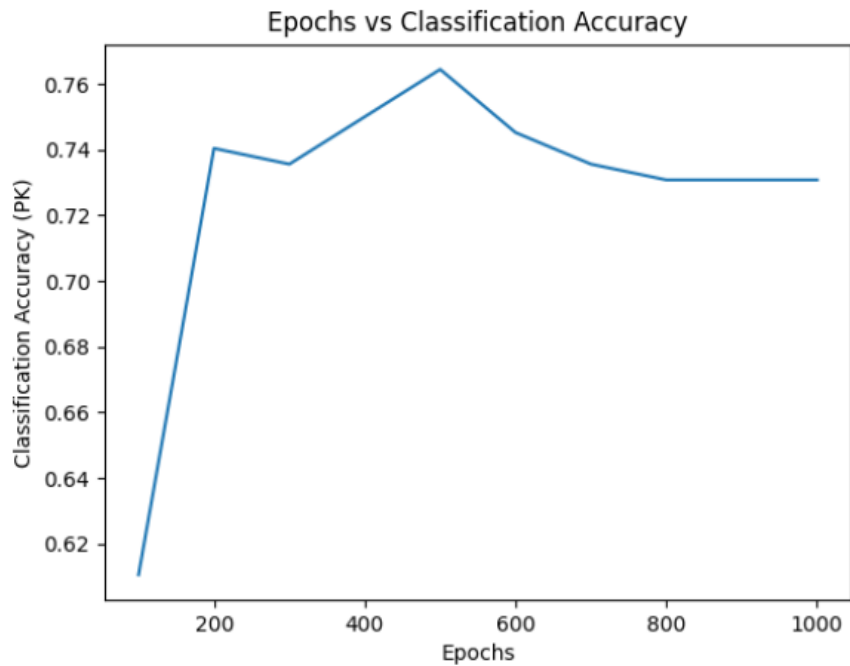
4.4 Eksperyment IV

W eksperymencie czwartym zbadano zależność między poprawnością klasyfikacji a liczbą epok. W tym celu przyjęto następujące parametry: dla wstecznej propagacji błędu $K1=80$, $K2=5$, dla obu algorytmów błąd został ustawiony jako 0.25, a współczynnikom uczenia nadano wartość 0.0001. Epoki zostały przypisane jako liczby zaczynając od 100 do 1000 z krokiem co 100 jednostek.



Rysunek 4.10 Wartość PK dla różnych epok we wstecznej propagacji błędu

Rysunek 4.10 przedstawia średnią wartość poprawności klasyfikacji przy różnej ilości epok, gdzie wykorzystany został algorytm wstecznej propagacji błędu. Widać na nim, że zwiększając liczbę epok od 300 do 800, wartość PK stale się poprawiała, natomiast w zakresie od 200 do 300 oraz dla zakresu od 800 do 900 epok można zauważyć, że wystąpił spadek poprawności klasyfikacji. Na przestrzeni całego przedziału, klasyfikacja miała wartości zaczynające się od 58% przy 100 epokach, do aż 74% poprawności przy 800 epokach.

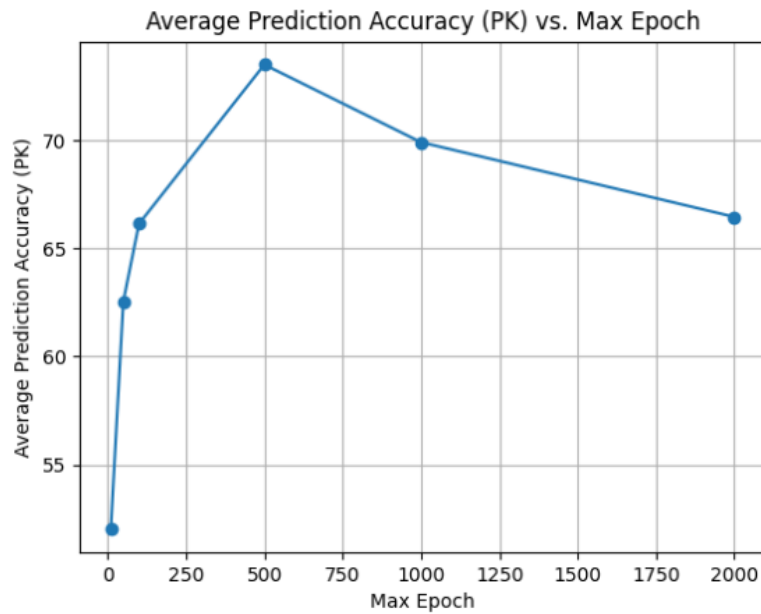


Rysunek 4.11 Wartość PK dla różnych epok w LVQ

W tym przypadku wykorzystany został algorytm LVQ, gdzie wartości dla PK zaczynały się od 61%, a kończyły na 76%. Dla wzrostu liczby epok można zauważyć poprawę klasyfikacji, począwszy od 100 epok do aż 500, wartość PK wzrasta, natomiast w zakresie od 200 do 400 oraz dla zakresu od 600 do 1000 epok widać lekki spadek poprawności, co przedstawia rysunek 4.11.

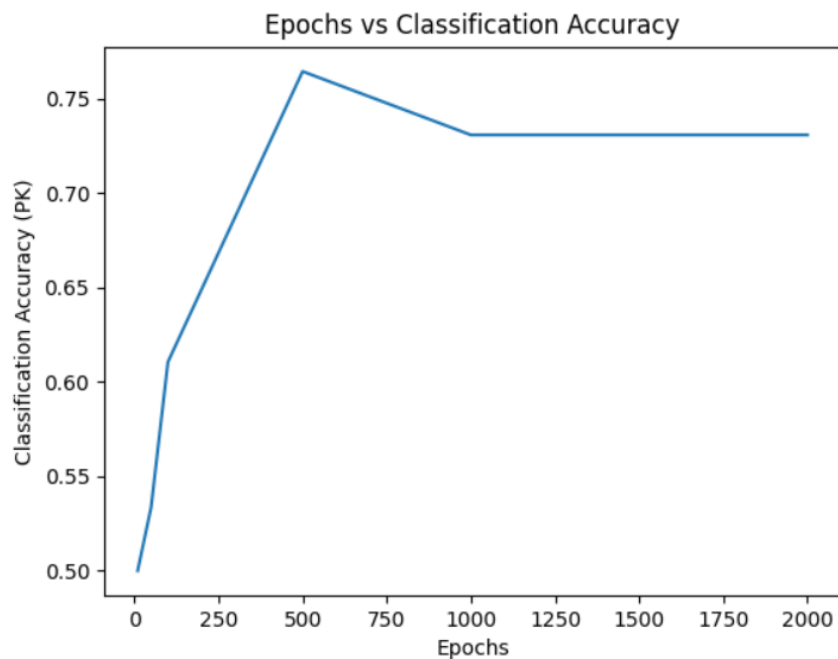
4.5 Eksperyment V

Eksperyment ten opierał się na większym zróżnicowaniu zakresu epok, których wartość wynosiła dokładnie [10, 50, 100, 500, 1000, 2000]. Współczynnik uczenia został przypisany jako 0.0001, wartości neuronów w warstwach pozostały niezmiennie, podobnie jak współczynniki błędów.



Rysunek 4.12 Wartość PK dla różnych epok we wstecznej propagacji błędu

W tym przypadku średnia wartość poprawności klasyfikacji mieściła się w przedziale od 50% do 75%. Najniższy z wyników został zaobserwowany przy 10 epokach, natomiast najwyższy wskaźnik PK został zaobserwowany przy 500 epokach. Od tej ilości epok, dla kolejnych iteracji współczynnik poprawności klasyfikacji zaczął spadać. Zależności te zostały ukazane na rysunku 4.12.



Rysunek 4.13 Wartość PK dla różnych epok w LVQ

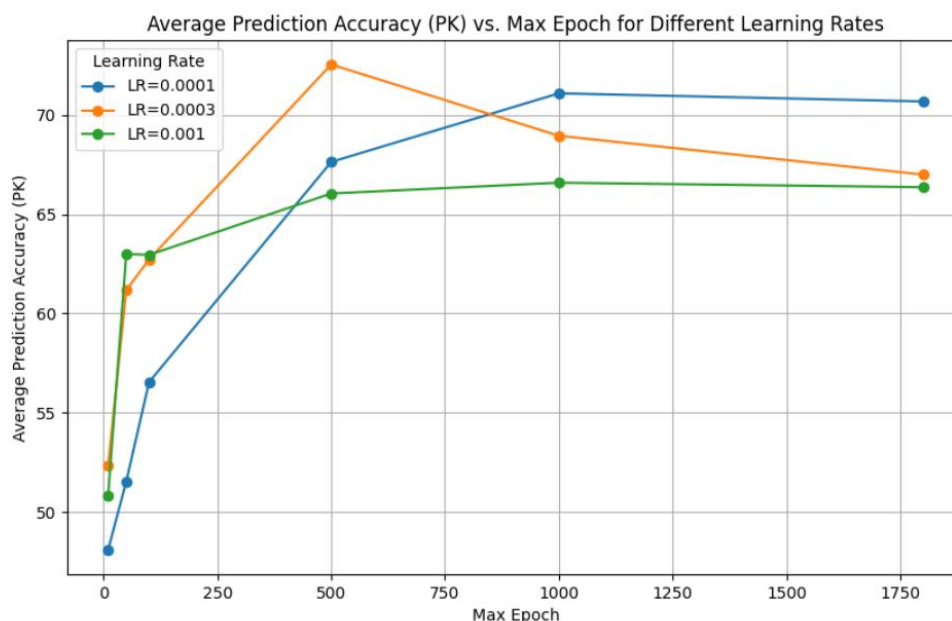
Na rysunku 4.13 można zauważyć, że dla algorytmu LVQ wartość PK zaczęła wzrastać wraz ze wzrostem ilości epok, a dokładniej w przedziale od 10 do 600 epok, a następnie spadać w przedziale od 600 epok do 1000. Różnica jest widoczna w przedziale od 1000 do 2000 epok, gdzie wartość współczynnika uczenia pozostała w przybliżeniu niezmienna. Na przestrzeni całego wykresu wartości PK mieściły się w przedziale od 50% do 76%.

Na obu wykresach (rysunek 4.13 i rysunek 4.12) widać podobieństwo, gdzie dla epok od 10 do 500 wartość PK wzrasta, natomiast dla epok od 500 do 1000 wartość ta maleje, a następnie w przybliżeniu utrzymuje się na podobnym poziomie.

Dla obu algorytmów zauważono, że pozytywnym zjawiskiem było zmniejszenie liczby epok do wartości 500. Zmniejszenie epok wskazuje na to, że model może uczyć się efektywnie w tym zakresie, co jest pozytywnym zjawiskiem, ponieważ oznacza to krótszy czas trenowania i mniejsze zużycie zasobów obliczeniowych. Jeżeli model trenuje się przez zbyt wiele epok, może wystąpić przetrenowanie sieci, a tym samym oznacza to pogorszenie wyników sieci.

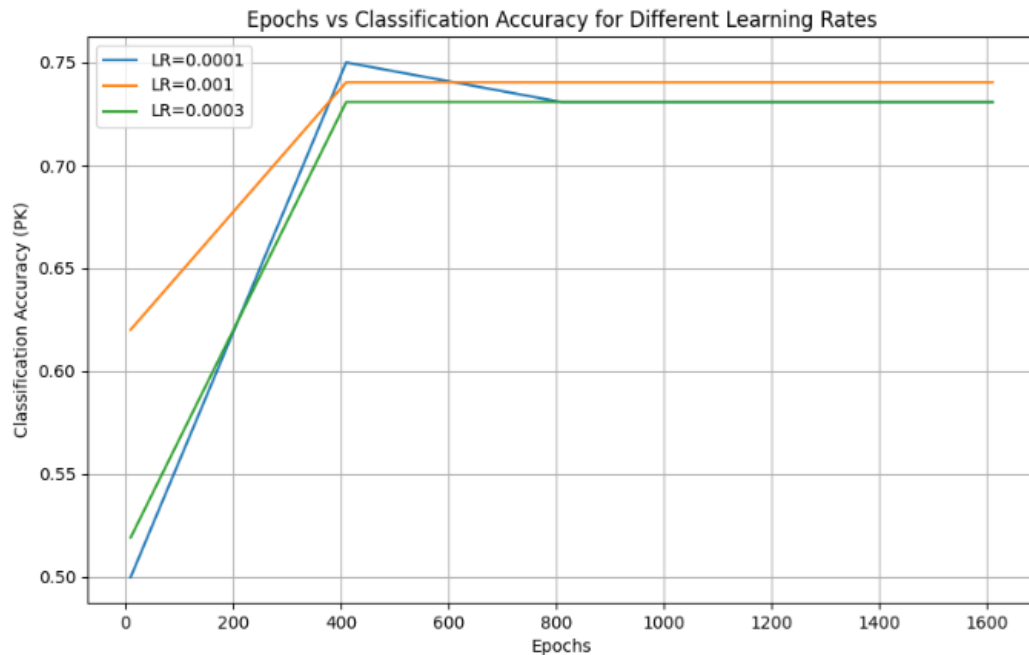
4.6 Eksperyment VI

Eksperyment szósty opierał się na zróżnicowanych wartościach epok i współczynnikach uczenia. Liczba neuronów w warstwach ukrytych dla algorytmu wstecznej propagacji błędów pozostała niezmienna, tak samo jak współczynnik błędów. Eksperyment ten miał na celu sprawdzenie poprawności klasyfikacji z różnymi wartościami epok oraz przy różnych współczynnikach uczenia.



Rysunek 4.14 Kolejne wartości PK w zależności od lr i epok dla wstecznej propagacji błędów

Wykres na rysunku 4.14 przedstawia kolejne wartości współczynnika PK przy różnych wartościach lr w kolejnych epokach. Zakres wartości mieści się między 45% a 75%, gdzie minimalna wartość została osiągnięta dla $lr=0.0001$ oraz $epoch=10$, natomiast najwyższą wartość osiągnięto przy $lr=0.001$ i $epoch=500$. Wartości od 1000 epok utrzymywały się w przybliżeniu na równym poziomie.



Rysunek 4.15 Kolejne wartości PK w zależności od lr i epoch dla LVQ

W przypadku sieci LVQ wartości PK znajdowały się w przedziale 50% do 75%, gdzie wartość minimalna została osiągnięta dla $lr=0.0001$ przy $epoch=10$, natomiast najwyższą wartość osiągnięto dla $lr=0.0001$ przy $epoch=400$. Wykres ten przedstawia rysunek 4.15, na którym widać także, że wartość PK jest w przybliżeniu niezmienna od epoki o wartości 800 i znajduje się na poziomie 72%. Jest to znak, że sieć osiągnęła punkt stabilizacji, gdzie dalsze trenowanie nie przynosi już znaczącej poprawy skuteczności klasyfikacji.

5 Podsumowanie i wnioski

Przeprowadzone eksperymenty koncentrowały się na dwóch głównych algorytmach: wstecznej propagacji błędów i Learning Vector Quantization (LVQ). Ich celem było zbadanie wpływu różnych parametrów na efektywność działania sieci neuronowych i porównanie ich wyników, wydajności i efektywności. Eksperyment pierwszy wykazał, że zarówno liczba neuronów w poszczególnych warstwach ukrytych, jak i ich odpowiednia konfiguracja, mają istotny wpływ na skuteczność klasyfikacji. Zaobserwowano, że większa liczba w pierwszej warstwie często prowadziła do lepszych wyników, jednak istotne było także dostosowanie liczby neuronów w

kolejnych warstwach. Badanie różnych kombinacji parametrów, takich jak liczba epok, współczynnik błędu i współczynnik uczenia pokazały, że optymalizacja tych parametrów jest niezbędna dla osiągnięcia najlepszych wyników. Warto zauważyć, że istnieje delikatna równowaga między tymi parametrami, a ich optymalna wartość może się różnić od konkretnego zestawu danych. Eksperymenty sugerują, że większa złożoność sieci, wyrażona przez większą liczbę neuronów, nie zawsze się przekłada na lepszą skuteczność klasyfikacji. W niektórych przypadkach mniejsza liczba neuronów może prowadzić do lepszych wyników. Obserwacje zmian błędu SSE wraz z kolejnymi epokami potwierdzają, że proces uczenia się sieci neuronowej przebiega zgodnie z oczekiwaniami, a wartość błędu maleje wraz z postępem uczenia. Jest to istotne zjawisko, które potwierdza skuteczność wybranych algorytmów uczenia. W trakcie eksperymentów można było zauważyć, że zarówno wsteczna propagacja błędu, jak i algorytm LVQ osiągają różne wyniki w zależności od ustawionych parametrów. W przypadku wstecznej propagacji błędu, najlepszy współczynnik uczenia wyniósł 0.0002, co skutkuje klasyfikacją na poziomie 72% przy 80 neuronach w warstwie pierwszej i 5 w drugiej. Warto zaznaczyć, że dla większych wartości współczynnika uczenia poprawność klasyfikacji zaczęła maleć, co sugeruje, że zbyt duże zmiany wag sieci mogą prowadzić do pogorszenia wyników. Dla algorytmu LVQ natomiast, najlepsza skuteczność była dla współczynnika uczenia równego 0.0001, osiągając maksymalną wartość około 74%. W kontekście liczby epok, dla wstecznej propagacji błędu zauważyć można, że poprawność klasyfikacji stale rośnie z liczbą epok do około 800 w eksperymencie trzecim, a także dla wartości 500 epok w eksperymencie czwartym, po czym następuje lekki spadek poprawności klasyfikacji. Dla LVQ również zaobserwowano wzrost skuteczności wraz ze wzrostem liczby epok do około 500, a następnie spadek poprawności. Podsumowując, eksperymenty koncentrujące się na tych dwóch algorytmach potwierdzają, że proces uczenia się sieci neuronowej przebiega zgodnie z oczekiwaniami, gdzie wartość błędu maleje wraz z postępem uczenia. Zarówno wsteczna propagacja błędu, jak i algorytm LVQ osiągają różne wyniki w zależności od ustawionych jednakowych parametrów. W przypadku wstecznej propagacji błędu, optymalny współczynnik uczenia wyniósł 0.0002, a dla LVQ skuteczność była najwyższa dla współczynnika 0.0001. Obserwowane zmiany poprawności klasyfikacji wraz z liczbą epok zasugerowały, że dobór odpowiedniej liczby epok również odgrywa dużą rolę w skuteczności działania obu algorytmów.

6 Bibliografia

- [1] <https://archive.ics.uci.edu/dataset/151/connectionist+bench+sonar+mines+vs+rocks>

Data dostępu: 14.05.2024

- [2] PRz, KIIA, Sztuczna Inteligencja, „Sieć neuronowa jednokierunkowa jednowarstwowa”, Roman Zajdel,

Data dostępu: 14.05.2024

- [3] PRz, KIIA, Sztuczna Inteligencja, „Sieć neuronowa jednokierunkowa wielowarstwowa”, Roman Zajdel,

Data dostępu: 14.05.2024

- [4] Kluska J. Wykład ze Sztucznej Inteligencji,

Data dostępu: 14.05.2024

- [5] <https://www.turing.com/kb/application-of-learning-vector-quantization>

Data dostępu: 21.05.2024

- [6] <https://www.geeksforgeeks.org/learning-vector-quantization/>

Data dostępu: 24.05.2024

- [7] <https://www.guru99.com/pl/backpropagation-neural-network.html>

Data dostępu: 14.05.2024

Spis rysunków

Rysunek 2.1 Przedstawienie ilości i rodzaju danych.....	5
Rysunek 3.1 Model neuronu	5
Rysunek 3.2 Sieć neuronowa jednowarstwowa	7
Rysunek 3.3 Sieć neuronowa wielowarstwowa	8
Rysunek 3.4 Przedstawienie propagacji wstecznej	9
Rysunek 3.5 Sieć LVQ.....	15
Rysunek 4.1 Zależność poprawności klasyfikacji od wyższej liczby neuronów	19
Rysunek 4.2 Wykres błędu SSE dla fazy 1 eksperymentu I	20
Rysunek 4.3 Zależność poprawności klasyfikacji od niższej liczby neuronów	21
Rysunek 4.4 Wykres błędu SSE dla fazy 2 eksperymentu I	22
Rysunek 4.5 Wpływ zróżnicowanej ilości neuronów na poprawność klasyfikacji.....	23
Rysunek 4.6 Wykres błędu SSE dla fazy 3 eksperymentu I	24

Rysunek 4.7 Wartość PK dla różnych współczynników l_r w LVQ	25
Rysunek 4.8 Wartość PK dla wstecznej propagacji błędu przy kolejnych l_r	26
Rysunek 4.9 Wartość PK dla LVQ przy kolejnych l_r	26
Rysunek 4.10 Wartość PK dla różnych epok we wstecznej propagacji błędu	27
Rysunek 4.11 Wartość PK dla różnych epok w LVQ.....	28
Rysunek 4.12 Wartość PK dla różnych epok we wstecznej propagacji błędu	29
Rysunek 4.13 Wartość PK dla różnych epok w LVQ.....	29
Rysunek 4.14 Kolejne wartości PK w zależności od l_r i epok dla wstecznej propagacji błędu	30
Rysunek 4.15 Kolejne wartości PK w zależności od l_r i epok dla LVQ.....	31

Spis listingów

Listing 2.1 Kod wczytujący i normalizujący dane	4
Listing 3.1 Wsteczna propagacja błędu	14
Listing 3.2 Learning Vector Quantization	17